C2 — Presentation 2 Prep: Technical Q&A Behind the Scenes

Panel comment being addressed: *"unable to explain technical aspects behind the scene."*

Everything below is written as short, speakable answers with exact code references, so you can point at the screen and say "here" instead of describing from memory.

---

1. How my component works (the 30-second pipeline)

C2 is the **Browser-in-the-Browser (BitB) phishing detector**. BitB = a page draws a *fake* browser window (fake address bar, fake padlock) inside the real browser to steal credentials.

Data flow when a page loads:

1. **Playwright Chromium** fires a `framenavigated` event -> `core/main.py` nav handler
2. The page's **DOM**, a **screenshot**, and **runtime signals** are extracted (`core/playwright_session.py`)
3. **Trust gate** first: if the eTLD+1 domain is in the 50,000-domain Tranco verified list -> verdict `VERIFIED`, risk 0, skip everything else (`core/c2/verified_domains.py`)
4. Otherwise **6 layers run concurrently** (`asyncio.gather` in `analyze()`):

- **L1 BitB DOM** — heuristics + XGBoost (`core/c2/layer1_bitb.py`)
- **L2 URL** — XGBoost + heuristic fallback (`core/c2/layer2_url.py`)
- **L3 Visual** — pHash screenshot vs brand-logo DB (`core/c2/layer3_visual.py`)
- **L4 Form** — cross-origin POST / password-field analysis (`core/c2/layer4_form.py`)
- **L5 Reputation** — Google Safe Browsing + PhishTank (`core/c2/layer5_reputation.py`)
- **L6 Runtime** — CDP event listeners + network observer + active keystroke probe (`core/c2/layer6_runtime.py`)

5. **Fusion** (`_fuse_score` in `core/main.py`): weighted sum `Risk = (L1×0.15 + L2×0.25 + L3×0.15 + L4×0.10 + L5×0.20 + L6×0.15) × 100`, plus a **decisive-signal floor**: L1 heuristic >= 0.9 -> PHISHING, >= 0.7 -> SUSPICIOUS
6. **Verdict**: SAFE / SUSPICIOUS (>=30) / PHISHING (>=60) / VERIFIED
7. If above threshold -> **interstitial overlay injected into the live page** (warn/block with "Continue anyway"), result broadcast over WebSocket to the dashboard, stored in the alert DB

One-sentence version: *"Every page the browser loads is scored in parallel by six independent detectors — DOM structure, URL, visuals, forms, reputation, runtime behaviour — fused into one 0–100 risk, and the user is warned inside the page itself before credentials can be typed."*

---

2. Where the AI models are imported in the code (point-and-show)

What	File : line	Code
L1 model loaded	`core/c2/layer1_bitb.py:27-32`	`pickle.load()` of `models/bitb_classifier.pkl` at import time
L1 16 features defined	`core/c2/layer1_bitb.py:43-49`	`_FEATURE_COLS` list

L1 feature extraction	`core/c2/layer1_bitb.py:91`	`_extract_html_features()` — same features as training
L1 inference	`core/c2/layer1_bitb.py:235-236`	`pd.DataFrame([feats])[_FEATURE_COLS]` -> `predict_proba(X)[0][1]`
L2 model loaded	`core/c2/layer2_url.py:20-23`	`pickle.load()` of `models/url_classifier.pkl`
L2 inference	`core/c2/layer2_url.py:149`	`_model.predict_proba(X)[0][1]`
L3 brand hash DB	`core/c2/layer3_visual.py:31`	`json.load()` of generated logo pHash database
Fusion weights/thresholds	`core/main.py _fuse_score()`	settings-driven, optional `c2_fusion.pkl` meta-model
Graceful fallback	both layers	`FileNotFoundError` -> heuristics only, app still runs

Training/tuning scripts (not shipped in the runtime, they're the offline pipeline):

Script	Job
`scripts/prepare_html_dataset.py`	build L1 dataset + train `bitb_classifier.pkl`
`scripts/prepare_dataset.py`	build L2 dataset + train `url_classifier.pkl`
`scripts/generate_logo_hashes.py`	generate L3 brand pHash database
`scripts/tune_models.py`	RandomizedSearchCV retuning of L1 + L2 (this session)
`scripts/evaluate_c2.py`	per-layer ROC/AUC + fused confusion matrix on 600 labelled pages
`scripts/capture_fusion_vectors.py` / `tune_fusion.py`	fusion meta-classifier experiment

---

3. Codebase architecture (one-liners per file)

```
R26-CS-003/
■-- core/
|   ■-- main.py                  FastAPI gateway: /analyze, nav handler, fusion, test panel SSE
|   ■-- playwright_session.py    Persistent Chromium: DOM/screenshot extraction, CDP runtime
|   |                            collection, interstitial injection, per-tab tracking
|   ■-- c2/                      ← MY COMPONENT
|       ■-- layer1_bitb.py       BitB DOM: 7 heuristic rules + XGBoost overlay + result cache
|       ■-- layer2_url.py        URL: XGBoost + heuristic fallback (free hosts, phish keywords)
|       ■-- layer3_visual.py     pHash screenshot vs 18 brand logos, >80% = impersonation
|       ■-- layer4_form.py       Form destination: cross-origin POST, password fields
|       ■-- layer5_reputation.py GSB + PhishTank, concurrent, TTL-cached
|       ■-- layer6_runtime.py    Runtime: keyloggers, clipboard hooks, off-origin exfil
|       ■-- verified_domains.py  Tranco trust gate (eTLD+1 + shared-host exclusion)
|       ■-- alert_store.py       SQLite alert/verdict persistence
|       ■-- reporter.py          Reporting helpers
■-- models/                      bitb_classifier.pkl, url_classifier.pkl (+ .bak backups)
■-- data/verified_domains.txt    50,000 verified domains
■-- frontend/dashboard.html      Electron dashboard: per-tab live cards, settings, Test panel
■-- scripts/                   Offline ML pipeline (table above)
■-- test/C2/                   Offline suites + realistic page fixtures (pages/, build_pages.py)
```

---

4. The AI models, explained

Why XGBoost (not a neural net)? Our inputs are small **tabular feature vectors** (16 DOM counts, 13 URL stats), not raw pixels/text. Gradient-boosted trees are the state of the art on tabular data at this scale: they train in seconds on ~80k pages, infer in microseconds (we run per navigation, so latency matters), are robust to unscaled/mixed features, and give feature importances we can defend. A deep net would be slower, need far more data, and add nothing here.

L1 input features (16): iframe count, fixed-position iframe, max z-index, full-viewport coverage, drag-prevention JS, form/input/password/hidden-input counts, external script count,

external form action, brand name in title, brand favicon, overlay element, redirect script, HTML size. *(These encode how a BitB overlay actually looks in the DOM.)*

L2 input features (13): URL length, digit/hyphen counts, brand-keyword presence, free-host TLD, IP-as-host, punycode, path depth, entropy-ish character stats, etc.

Hybrid design — `score = max(heuristic, ML)` **with ML capped (**`_ML_MAX_BOOST = 0.15`**):** the deterministic rules encode *known* attack signatures (so we can say exactly *why* a page was flagged — explainability), while the ML catches variations the rules miss. The cap means the ML can only add up to 0.15 on top of heuristics — it can never single-handedly convict a page. This was a deliberate design decision after finding the model is brittle out-of-distribution.

L3 is not a trained model — it's perceptual hashing (pHash): screenshots are hashed and compared by Hamming distance against 18 brand logos; >80% similarity flags impersonation.

---

5. AI tuning story (what I actually did this phase)

1. **Baseline evaluation harness** (`evaluate_c2.py`): 600 labelled pages from the Mendeley phishing corpus (300 phishing / 300 legit, `index.sql` labels) -> per-layer ROC/AUC + fused confusion matrix. Found: **nothing ever reached the PHISHING threshold** in static mode — quantified the hand-set weight/threshold problem.
2. **Per-model tuning** (`tune_models.py`): `RandomizedSearchCV` over XGBoost hyperparameters (max_depth, n_estimators, learning_rate, subsample…), **stratified 3-fold CV, F1-scored**; decision threshold picked from the **precision-recall curve**, not the default 0.5.
   Results: L1 test AUC **0.9577 -> 0.9637**, L2 **0.9075 -> 0.9022** (L2 traded a little AUC for better F1 at its operating point — deliberate).
3. **Fusion tuning experiment** (`capture_fusion_vectors.py` + `tune_fusion.py`): rendered 604 labelled pages headless, captured 6-layer score vectors, trained a **logistic meta-classifier -> AUC 0.9672 vs 0.8468 **for the hand-set weights.** But it zeroed out L6/L3** — because batch-rendered saved pages can't reproduce live runtime behaviour (origin `about:blank`, external assets don't load). **Decision: NOT applied** — applying it would make production ignore the runtime layer. Correct next step is logging vectors from live navigation.
4. **Validation against real attack tooling**: tested the actual **mrd0x BitB kits** (public red-team templates) — heuristics scored only 0.35 -> added R6/R7 signatures -> **0.95**, and added the decisive-signal floor so a perfect DOM detection can't be diluted below PHISHING by a clean URL.

---

6. Likely panel questions — rehearsed answers

Q: Where exactly is the AI in your system?

A: Two XGBoost classifiers, loaded as pickles — L1 at `layer1_bitb.py:27`, L2 at `layer2_url.py:20`, inference via `predict_proba`. Plus pHash for the visual layer. The other layers are deterministic heuristics and external reputation APIs.

Q: What data did you train on?

A: The Mendeley phishing webpage corpus — ~80k real HTML snapshots with SQL labels (phishing=1/legit=0). L1 trains on the 16 DOM features extracted from those pages; L2 on the 13 URL features from their URLs.

Q: How did you prevent overfitting?

A: Stratified 3-fold cross-validation inside RandomizedSearchCV, held-out test AUC reported separately, F1-based model selection (not accuracy — accuracy hides the minority class), and the decision threshold chosen on the PR curve.

Q: Why F1 and not accuracy?

A: Phishing detection is a precision/recall trade-off; on imbalanced real traffic a model can get 99% accuracy by calling everything safe. F1 balances false alarms against missed attacks.

Q: How do you handle false positives?

A: Three mechanisms: (1) the verified-domain trust gate (50k Tranco domains -> VERIFIED, risk 0); (2) the ML overlay is capped at +0.15 over heuristics; (3) a dedicated realistic-benign test suite (bank login with fixed header, modal, same-origin form — every FP trap) that must stay SAFE. Verified live: `google.com` -> VERIFIED, benign login -> SAFE 18.7/100.

Q: Can the ML model alone convict a page?

A: No — by design. The ML overlay is capped at +0.15 over the deterministic heuristic score, and the fusion floor keys off the heuristic sub-score, not the ML output. The ML refines; rules decide.

Q: Weaknesses of your model? *(be honest — panels reward this)*

A: The L1 model is brittle out-of-distribution — a realistic benign login page it never saw in training can score high. In production this is masked by the trust gate and L1's 0.15 fusion weight, and the floor ignores the ML score. The proper fix is retraining with realistic legit login pages and real BitB kits in the corpus — that's planned work.

Q: Why is your learned fusion not in production?

A: It achieved AUC 0.9672 offline, but it learned to zero out the runtime layer — because our capture method (batch-rendered saved pages) can't reproduce live runtime behaviour. Applying it would make production *worse*. So we kept the configurable weighted sum and documented the correct procedure (live-navigation vector logging) as future work.

Q: Real-time performance?

A: Layers run concurrently (~44 ms for a full 6-layer analysis), per-layer content-hash caching drops repeat pages to ~0.1 ms, reputation lookups are TTL-cached. The browser never blocks on analysis.

Q: What happens if the model file is missing/corrupt?

A: Graceful degradation — each layer catches `FileNotFoundError` at load and falls back to pure heuristics; the system still runs, just without the ML overlay.

Q: How is this different from Google Safe Browsing alone?

A: GSB (our L5) is a blocklist — it only knows *reported* sites and lags new campaigns. C2

detects the *attack technique itself* from the page's structure and behaviour, so it catches zero-hour BitB pages on never-seen-before or even compromised-legitimate domains — our compromised-domain test proves a completely clean URL still gets PHISHING.

Q: How did you test against real attacks?

A: We used the public mrd0x BitB kit templates (the reference red-team implementation), built self-contained pages from them, rendered them in the live browser, and ran the full pipeline. That exposed a real gap (score 0.35) which we fixed with two additive signatures (0.95).

Q: Reproducibility?

A: The full pipeline is scripted: dataset prep -> train -> tune -> evaluate, all under `scripts/`, with metrics/ROC/confusion artifacts committed under `researches/c2/artifacts/`. Old models are backed up as `*.pkl.bak`.

---

7. Numbers cheat-sheet (memorize these)

Metric	Value
L1 test AUC (after tuning)	**0.9637**
L2 test AUC (after tuning)	**0.9022**
Fused AUC (static evaluation, 600 pages)	**0.9693**
Learned fusion AUC (not applied)	0.9672 vs 0.8468 baseline
Fusion weights	L1 .15 · L2 .25 · L3 .15 · L4 .10 · L5 .20 · L6 .15
Verdicts	SAFE < 30 <= SUSPICIOUS < 60 <= PHISHING; VERIFIED = trusted
Floor	L1 heuristic >= 0.9 -> PHISHING; >= 0.7 -> SUSPICIOUS
Real mrd0x kit detection	0.35 -> **0.95**
Live Test-panel	**14/14 pass**, step-mode with 38 narrated pauses
Full analysis latency	~44 ms (concurrent), ~0.1 ms cached
Training corpus	Mendeley phishing webpages (~80k pages, SQL labels)